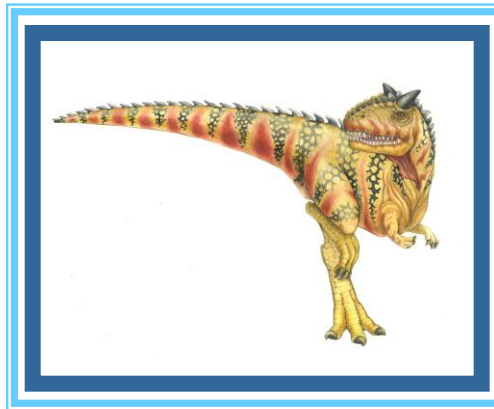
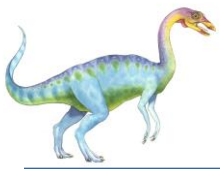


Chapter 12: Mass-Storage Systems





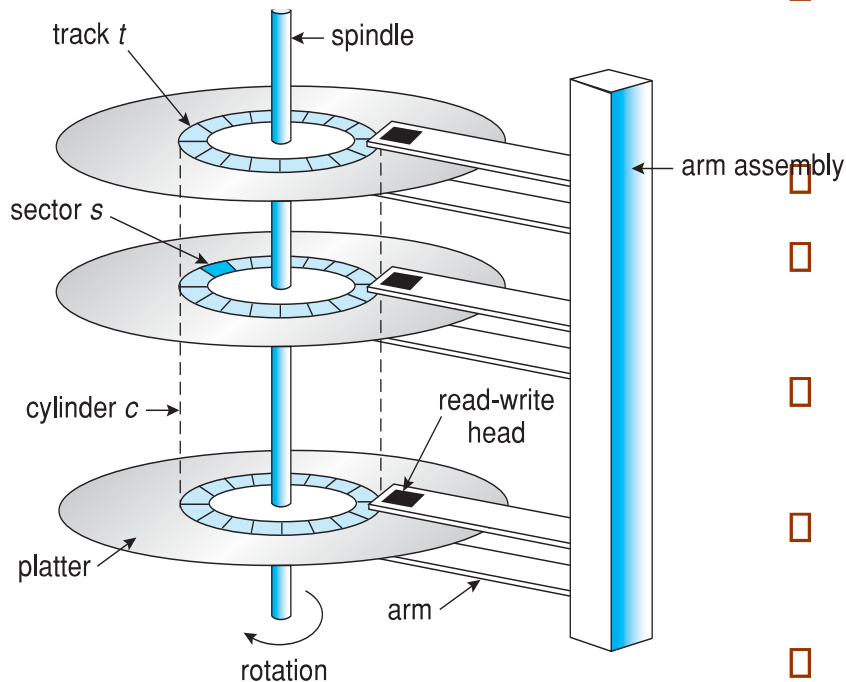
Chapter 10: Mass-Storage Systems

- Disk Structure
- Disk Scheduling
- Swap-Space Management

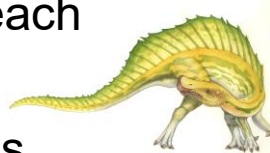


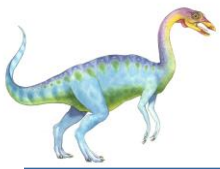


Magnetic Disk



- ❑ Magnetic disks provide the bulk of mass storage for modern computer systems.
- ❑ diameters range from 1.8 to 5.25 inches
- ❑ The two surfaces of a platter are covered with a magnetic material.
- ❑ information is stored by recording it magnetically on the platters
- ❑ A read–write head “flies” just above each surface of every platter.
- ❑ The heads are attached to a **disk arm** that moves all the heads as a unit.
- ❑ The surface of a platter is logically divided into circular tracks, which are subdivided into sectors.
- ❑ The set of tracks that are at one arm position makes up a cylinder.
- ❑ There may be thousands of concentric cylinders in a disk drive, and each track may contain hundreds of sectors.
- ❑ The storage capacity of common disk drives is measured in gigabytes.





Overview of Mass Storage Structure

- ❑ When the disk is in use, a drive motor spins it at high speed
- ❑ Drives rotate at 60 to 250 times per second
- ❑ **Transfer rate** is rate at which data flow between drive and computer
- ❑ **Positioning time (random-access time)** is
 - ❑ time to move disk arm to desired cylinder (**seek time**)
 - ❑ time for desired sector to rotate under the disk head (**rotational latency**)
- ❑ Disks can be removable
- ❑ Drive attached to computer via **I/O bus**
 - ❑ Several kinds of buses are available, including **enhanced integrated drive electronics (EIDE), advanced technology attachment (ATA), serial ATA (SATA), universal serial bus (USB), fiber channel (FC), and small computer-systems interface (SCSI) buses.**
- ❑ The data transfers on a bus are carried out by special electronic processors called **controllers**.
- ❑ The **host controller** is the controller at the computer end of the bus.
- ❑ A **disk controller** is built into each disk drive





- ❑ To perform a disk I/O operation, the computer places a command into the host controller
- ❑ The host controller then sends the command via messages to the disk controller,
- ❑ disk controller operates the disk-drive hardware to carry out the command.
- ❑ Disk controllers usually have a built-in cache.
- ❑ Data transfer at the disk drive happens between the cache and the disk surface
- ❑ Data transfer to the host, at fast electronic speeds, occurs between the cache and the host controller.





Disk Structure

- ❑ Disk drives are addressed as large 1-dimensional arrays of **logical blocks**, where the logical block is the smallest unit of transfer
- ❑ The size of a logical block is usually 512 bytes, although some disks can be **low-level formatted** to have a different logical block size, such as 1,024 bytes.
- ❑ The 1-dimensional array of logical blocks is mapped into the sectors of the disk sequentially
 - ❑ Sector 0 is the first sector of the first track on the outermost cylinder
 - ❑ Mapping proceeds in order through that track, then the rest of the tracks in that cylinder, and then through the rest of the cylinders from outermost to innermost
- ❑ Logical to physical address should be easy
 - ❑ difficult to perform this translation, for two reasons.
 - ❑ First, most disks have some defective sectors, but the mapping hides this by substituting spare sectors from elsewhere on the disk.
 - ❑ Second, the number of sectors per track is not a constant on some drives





Disk Scheduling

- The operating system is responsible for using hardware efficiently — for the disk drives, this means having a fast access time and disk bandwidth
- **Minimize seek time-** time taken for the disk arm to move the heads to the cylinder containing the desired sector.
- Seek time $\approx$ seek distance
- **Rotational latency** is the additional time for the disk to rotate the desired sector to the disk head
- **Disk bandwidth** is the total number of bytes transferred, divided by the total time between the first request for service and the completion of the last transfer
- can improve both the access time and the bandwidth by managing the order in which disk I/O requests are serviced





Disk Scheduling (Cont.)

- Whenever a process needs I/O to or from the disk, it issues a system call to the operating system.
- The request specifies several pieces of information:
 - Whether this operation is input or output
 - What the disk address for the transfer is
 - What the memory address for the transfer is
 - What the number of sectors to be transferred is
- OS maintains queue of requests, per disk or device





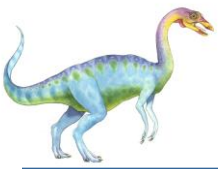
Disk Scheduling (Cont.)

- If the desired disk drive and controller are available, the request can be serviced immediately.
- If the drive or controller is busy, any new requests for service will be placed in the queue of pending requests for that drive.
- For a multiprogramming system with many processes, the disk queue may often have several pending requests.
- Thus, when one request is completed, the operating system chooses which pending request to service next.
- How does the operating system make this choice?
- Any one of several disk-scheduling algorithms can be used,
- scheduling algorithms with a request queue (0-199)

98, 183, 37, 122, 14, 124, 65, 67

Head pointer 53



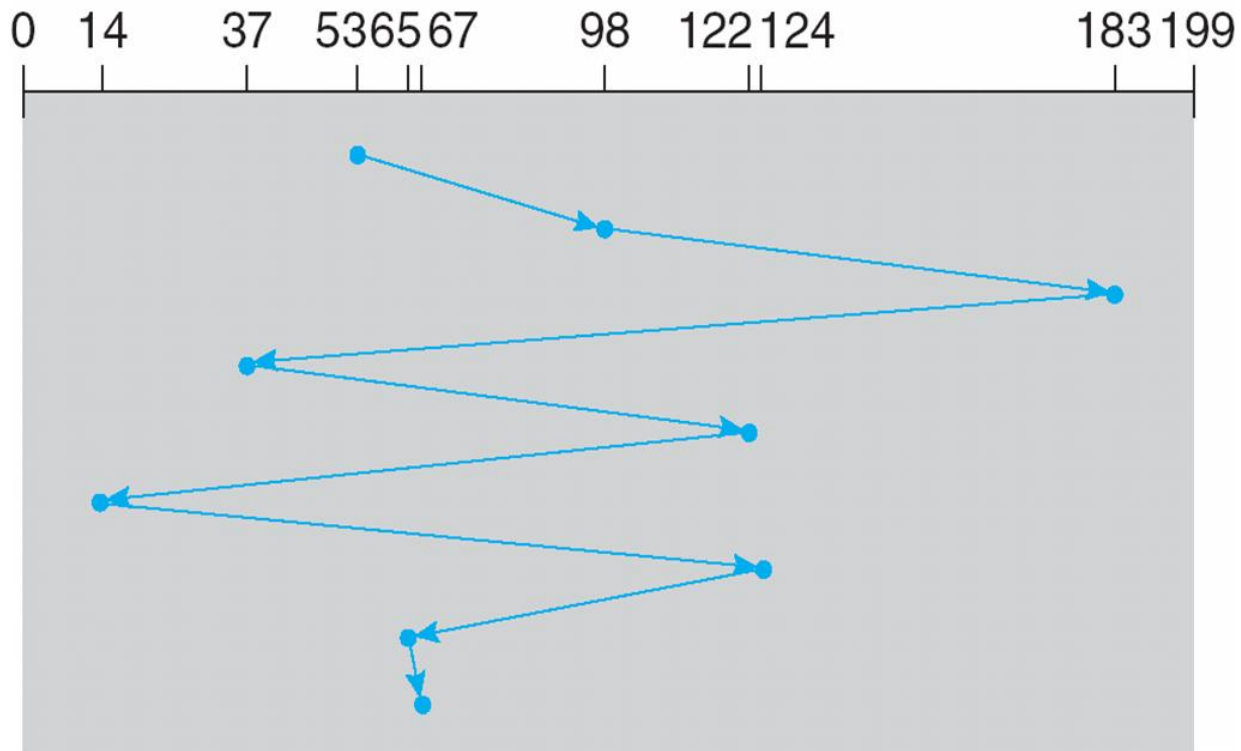


FCFS

Illustration shows total head movement of 640 cylinders

queue = 98, 183, 37, 122, 14, 124, 65, 67

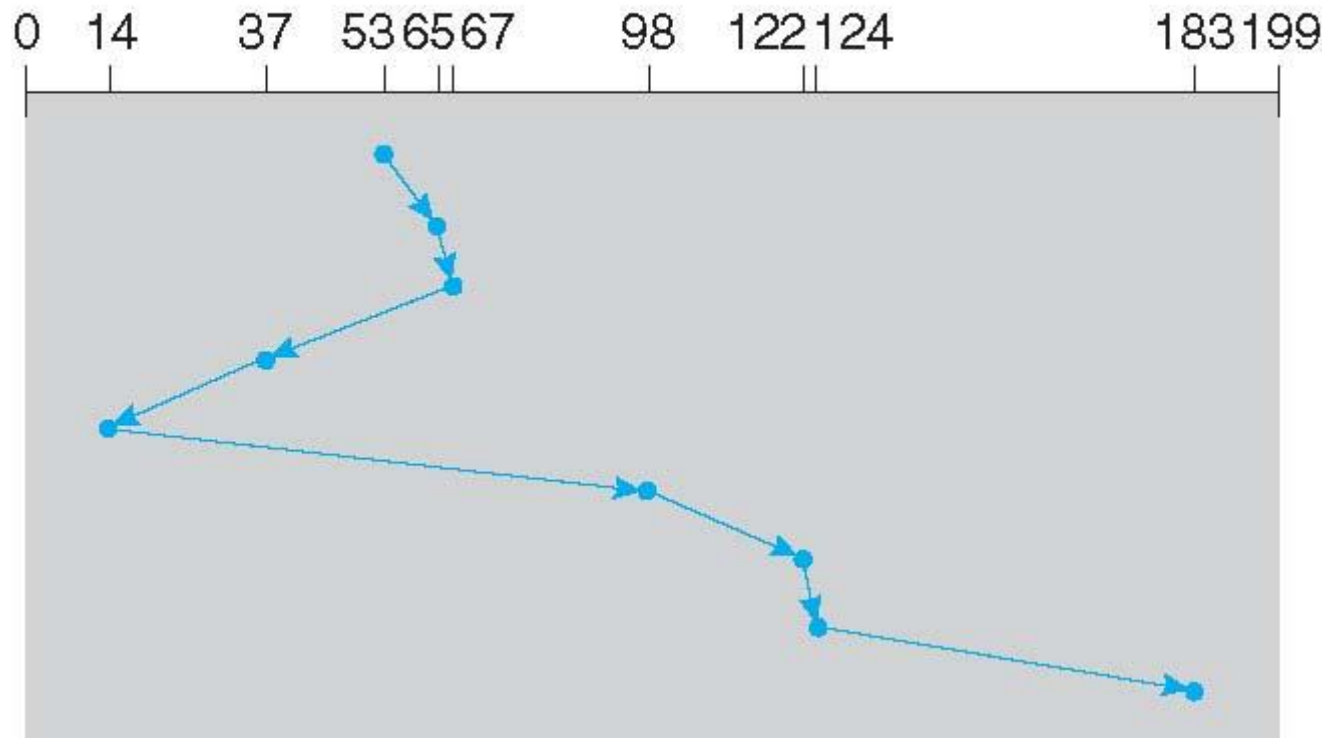
head starts at 53

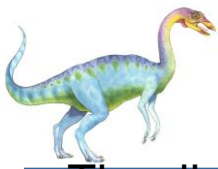




SSTF

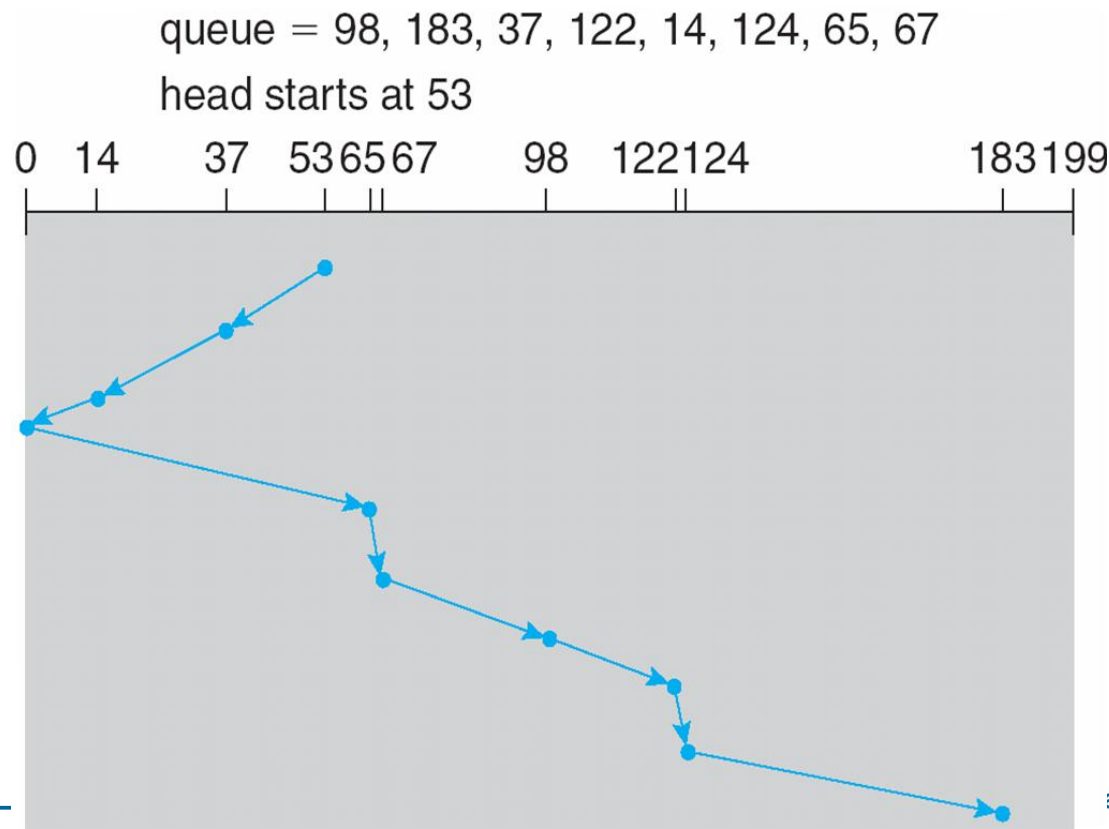
- Shortest Seek Time First selects the request with the minimum seek time from the current head position
- SSTF scheduling is a form of SJF scheduling; may cause starvation of some requests
- Illustration shows total head movement of 236 cylinders
queue = 98, 183, 37, 122, 14, 124, 65, 67
head starts at 53

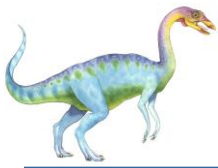




SCAN

- ❑ The disk arm starts at one end of the disk, and moves toward the other end, servicing requests until it gets to the other end of the disk, where the head movement is reversed and servicing continues.
- ❑ **SCAN algorithm**-called the **elevator algorithm**
- ❑ Illustration shows total head movement of 236 cylinders
- ❑ If requests are uniformly dense, largest density at other end of disk and those wait the longest





C-SCAN

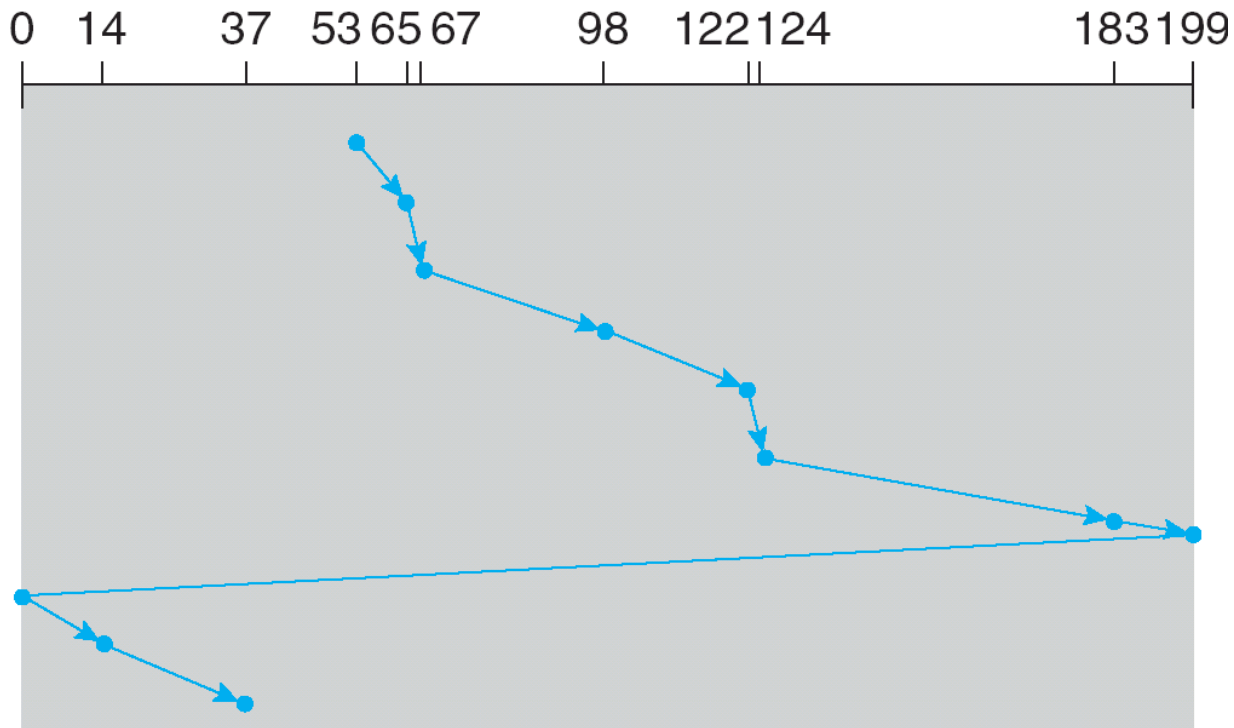
- Provides a more uniform wait time than SCAN
- The head moves from one end of the disk to the other, servicing requests as it goes
 - When it reaches the other end, it immediately returns to the beginning of the disk, without servicing any requests on the return trip
- Treats the cylinders as a circular list that wraps around from the last cylinder to the first one
- Total number of cylinders?

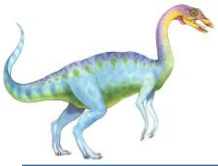




C-SCAN (Cont.)

queue = 98, 183, 37, 122, 14, 124, 65, 67
head starts at 53





C-LOOK

- ❑ LOOK a version of SCAN, C-LOOK a version of C-SCAN
- ❑ Arm only goes as far as the last request in each direction, then reverses direction immediately, going all the way to the end of the disk
- ❑ Total number of cylinders?

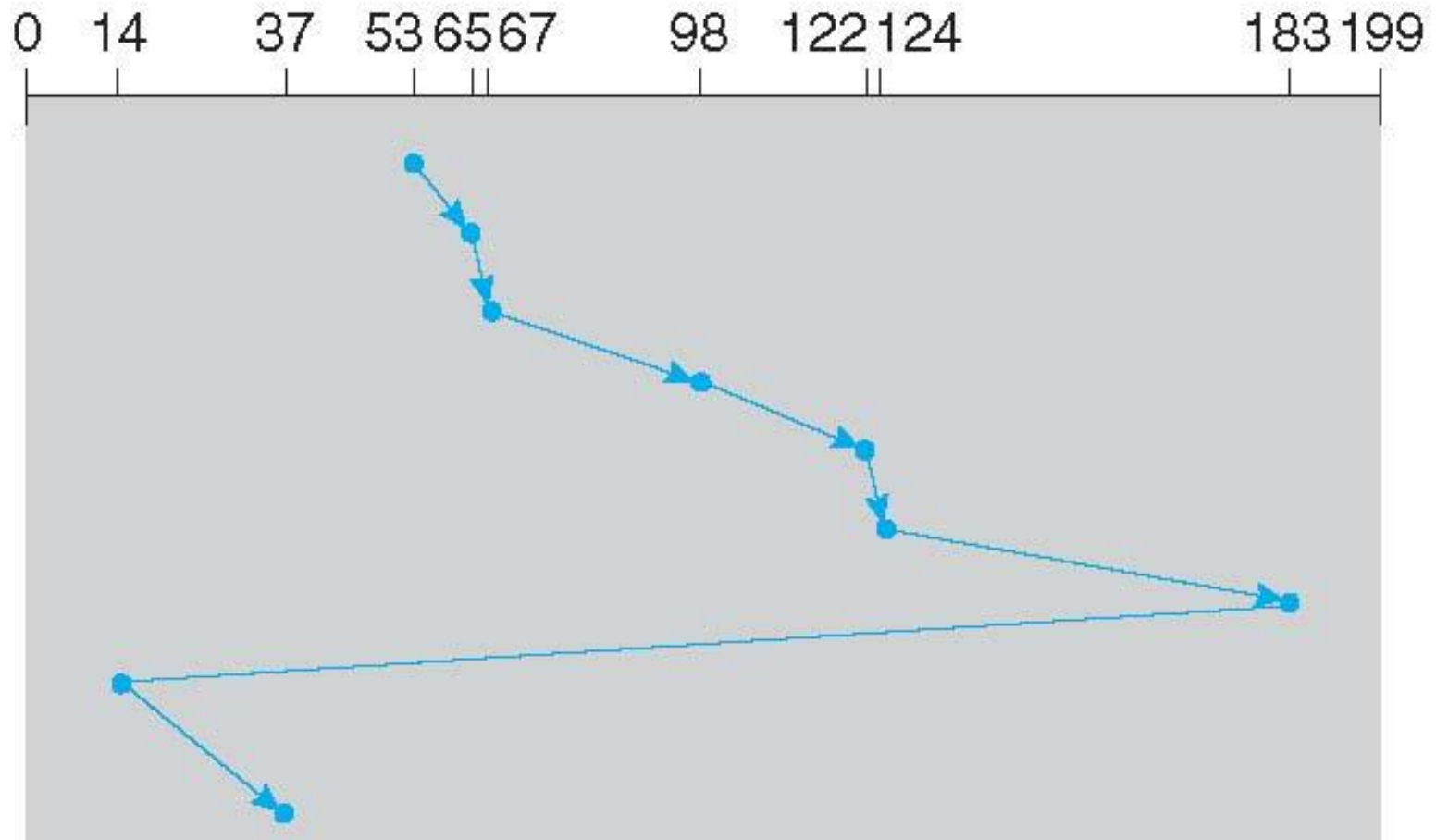




C-LOOK (Cont.)

queue = 98, 183, 37, 122, 14, 124, 65, 67

head starts at 53





Selecting a Disk-Scheduling Algorithm

- ❑ SSTF is common and has a natural appeal
- ❑ SCAN and C-SCAN perform better for systems that place a heavy load on the disk
 - ❑ Less starvation
- ❑ Performance depends on the number and types of requests
- ❑ Requests for disk service can be influenced by the file-allocation method
- ❑ The disk-scheduling algorithm should be written as a separate module of the operating system, allowing it to be replaced with a different algorithm if necessary
- ❑ Either SSTF or LOOK is a reasonable choice for the default algorithm





Swap-Space Management

- ❑ Swap-space — Virtual memory uses disk space as an extension of main memory
- ❑ disk access slower than memory access, -using swap space significantly decreases system performance.
- ❑ The main goal -to provide the best throughput for the virtual memory system.





Swap-Space Use

- used in various ways by different operating systems, depending on the memory-management algorithms in use.
- systems that implement swapping may use swap space to hold an entire process image, including the code and data segments.
- Paging systems may simply store pages that have been pushed out of mainmemory.
- The amount of swap space needed on a system can therefore vary from a few megabytes of disk space to gigabytes depending on the
 - amount of physical memory,
 - the amount of virtual memory it is backing,
 - and the way in which the virtual memory is used





Swap-Space Location

- ❑ can reside in one of two places: it can be carved out of the normal file system, or it can be in a separate disk partition.
- ❑ If the swap space is simply a large file within the file system, normal file-system routines can be used to create it, name it, and allocate its space.
- ❑ This approach, though easy to implement, is inefficient.
- ❑ Navigating the directory structure and the disk allocation data structures takes time and (possibly) extra disk accesses.
- ❑ External fragmentation can greatly increase swapping times by forcing multiple seeks during reading or writing of a process image.
- ❑ We can improve performance by caching the block-location information in physical memory and by using special tools to allocate physically contiguous blocks for the swap file, but the cost of traversing the file-system data structures remains





- ❑ swap space can be created in a separate **raw** partition.
- ❑ No file system or directory structure is placed in this space.
- ❑ Rather, a separate swap-space storage manager is used to allocate and deallocate the blocks from the raw partition.
- ❑ This manager uses algorithms optimized for speed rather than for storage efficiency, because swap space is accessed much more frequently than file systems (when it is used).
- ❑ Internal fragmentation may increase, but this trade-off is acceptable because the life of data in the swap space generally is much shorter than that of files in the file system.
- ❑ Since swap space is reinitialized at boot time, any fragmentation is short-lived.
- ❑ The raw-partition approach creates a fixed amount of swap space during disk partitioning.
- ❑ Adding more swap space requires either repartitioning the disk (which involves moving the other file-system partitions or destroying them and restoring them from backup) or adding another swap space elsewhere





Reference

- A. Silberschatz, P. B. Galvin and G. Gagne, *Operating System Concepts*, (9e), Wiley and Sons (Asia) Pvt. Ltd, 2016.

